

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

```
In [2]: plt.style.use('default')
sns.set_palette("husl")
```

```
In [3]: # Task 1: Joining the Datasets
print("\nTask 1")
print("-" * 30)

# Load the datasets
chinese_data = pd.read_csv('Tsang-2018-LexicalDecision.tsv', sep='\t')
affective_data = pd.read_csv('Mohammad-2018-AffectiveRatings.tsv', sep='\t')

print(f"Chinese lexical decision data: {chinese_data.shape[0]} rows, {chinese_data.shape[1]} columns")
print(f"Affective ratings data: {affective_data.shape[0]} rows, {affective_data.shape[1]} columns")

# Merge datasets on Concepticon concept ID
merged_data = pd.merge(chinese_data, affective_data, on='CONCEPTICON_ID', how='left')
print(f"Merged dataset: {merged_data.shape[0]} rows, {merged_data.shape[1]} columns")
print(f"Available columns: {list(merged_data.columns)}")

# Check that the chinese lemma exists
if 'CHINESE_LEMMA' not in merged_data.columns:
    print("Warning: CHINESE_LEMMA column not found. Check column names in Chinese dataset")
    print("Chinese dataset columns:", list(chinese_data.columns))
```

Task 1

```
-----
Chinese lexical decision data: 1109 rows, 7 columns
Affective ratings data: 2343 rows, 7 columns
Merged dataset: 957 rows, 13 columns
Available columns: ['CONCEPTICON_ID', 'CONCEPTICON_GLOSS_x', 'CHINESE', 'CHINESE_RT_MEAN', 'CHINESE_RT_ZSCORE', 'CHINESE_STROKE', 'LINE_IN_SOURCE_x', 'CONCEPTICON_GLOSS_y', 'ENGLISH', 'ENGLISH_VALENCE_MEAN', 'ENGLISH_AROUSAL_MEAN', 'ENGLISH_DOMINANCE_MEAN', 'LINE_IN_SOURCE_y']
Warning: CHINESE_LEMMA column not found. Check column names in Chinese dataset.
Chinese dataset columns: ['CONCEPTICON_ID', 'CONCEPTICON_GLOSS', 'CHINESE', 'CHINESE_RT_MEAN', 'CHINESE_RT_ZSCORE', 'CHINESE_STROKE', 'LINE_IN_SOURCE']
```

```
In [4]: # Task 2a: Create quintiles for the three main variables
print("Task 2a")

merged_data['stroke_quintile'] = pd.qcut(merged_data['CHINESE_STROKE'], 5, labels=True)
merged_data['arousal_quintile'] = pd.qcut(merged_data['ENGLISH_AROUSAL_MEAN'], 5, labels=True)
merged_data['rt_quintile'] = pd.qcut(merged_data['CHINESE_RT_MEAN'], 5, labels=True)

print("Quintile distributions:")
print("Stroke quintiles:")
print(merged_data['stroke_quintile'].value_counts().sort_index())
```

```
print("\nArousal quintiles:")
print(merged_data['arousal_quintile'].value_counts().sort_index())
print("\nRT quintiles:")
print(merged_data['rt_quintile'].value_counts().sort_index())

# Task 2b: Cross-tabulate combinations of variables
print("\nTask 2b: Cross-tabulation of variable pairs")

# Cross-tabulation 1: Stroke vs Arousal
crosstab_stroke_arousal = pd.crosstab(merged_data['stroke_quintile'], merged_data['arousal_quintile'])
print("\nStroke Quintiles vs Arousal Quintiles:")
print(crosstab_stroke_arousal)

# Cross-tabulation 2: Stroke vs RT
crosstab_stroke_rt = pd.crosstab(merged_data['stroke_quintile'], merged_data['rt_quintile'])
print("\nStroke Quintiles vs RT Quintiles:")
print(crosstab_stroke_rt)

# Cross-tabulation 3: Arousal vs RT
crosstab_arousal_rt = pd.crosstab(merged_data['arousal_quintile'], merged_data['rt_quintile'])
print("\nArousal Quintiles vs RT Quintiles:")
print(crosstab_arousal_rt)
```

Task 2a

Quintile distributions:

Stroke quintiles:

stroke_quintile

Q1 229

Q2 190

Q3 190

Q4 179

Q5 168

Name: count, dtype: int64

Arousal quintiles:

arousal_quintile

Q1 193

Q2 192

Q3 191

Q4 190

Q5 191

Name: count, dtype: int64

RT quintiles:

rt_quintile

Q1 192

Q2 191

Q3 191

Q4 191

Q5 192

Name: count, dtype: int64

Task 2b: Cross-tabulation of variable pairs

Stroke Quintiles vs Arousal Quintiles:

arousal_quintile	Q1	Q2	Q3	Q4	Q5
stroke_quintile					
Q1	41	45	55	48	40
Q2	42	41	32	40	35
Q3	31	38	37	44	40
Q4	40	32	32	39	36
Q5	39	36	35	18	40

Stroke Quintiles vs RT Quintiles:

rt_quintile	Q1	Q2	Q3	Q4	Q5
stroke_quintile					
Q1	44	62	40	35	48
Q2	47	40	44	32	27
Q3	47	36	37	40	30
Q4	37	26	37	39	40
Q5	17	27	33	45	46

Arousal Quintiles vs RT Quintiles:

rt_quintile	Q1	Q2	Q3	Q4	Q5
arousal_quintile					
Q1	44	35	42	29	43
Q2	38	38	33	48	35
Q3	34	40	36	37	44
Q4	42	38	42	33	35

Q5 34 40 38 44 35

The cross tabulation reveals patterns in the joint distribution of variables.

Deviations from uniform distribution suggest potential relationships between variables

```
In [5]: # Task 2c: Hierarchical cross-tabulation
print("\nTask 2c")

hierarchical_crosstab = pd.crosstab(
    [merged_data['stroke_quintile'], merged_data['arousal_quintile']],
    merged_data['rt_quintile']
)
print("\nHierarchical Cross-tabulation (Stroke & Arousal vs RT):")
print(hierarchical_crosstab)
```

Task 2c

Hierarchical Cross-tabulation (Stroke & Arousal vs RT):

rt_quintile		Q1	Q2	Q3	Q4	Q5
stroke_quintile	arousal_quintile					
Q1	Q1	10	12	6	3	10
	Q2	9	15	6	7	8
	Q3	6	14	11	11	13
	Q4	11	13	9	5	10
	Q5	8	8	8	9	7
Q2	Q1	10	9	11	4	8
	Q2	9	8	9	8	7
	Q3	13	6	5	5	3
	Q4	10	9	12	3	6
	Q5	5	8	7	12	3
Q3	Q1	3	7	9	5	7
	Q2	10	6	5	8	9
	Q3	10	6	10	7	4
	Q4	12	7	10	10	5
	Q5	12	10	3	10	5
Q4	Q1	16	2	7	8	7
	Q2	4	6	7	12	3
	Q3	3	7	5	6	11
	Q4	8	5	7	11	8
	Q5	6	6	11	2	11
Q5	Q1	5	5	9	9	11
	Q2	6	3	6	13	8
	Q3	2	7	5	8	13
	Q4	1	4	4	4	5
	Q5	3	8	9	11	9

```
In [6]: # Task 3a: Group by stroke count and compute averages
print("Task 3a")

stroke_grouped = merged_data.groupby('CHINESE_STROKE').agg({
    'ENGLISH_AROUSAL_MEAN': 'mean',
    'CHINESE_RT_MEAN': 'mean'
}).reset_index()

print("Average arousal and RT by stroke count:")
```

```
print(stroke_grouped)

# Task 3b: Create grouped barplot
print("\nTask 3b")

# Scale arousal by 1000
stroke_grouped['AROUSAL_SCALED'] = stroke_grouped['ENGLISH_AROUSAL_MEAN'] *

# Create the grouped barplot
fig, ax = plt.subplots(figsize=(12, 6))

x = np.arange(len(stroke_grouped))
width = 0.35

bars1 = ax.bar(x - width/2, stroke_grouped['AROUSAL_SCALED'], width,
               label='Arousal (×1000)', alpha=0.8)
bars2 = ax.bar(x + width/2, stroke_grouped['CHINESE_RT_MEAN'], width,
               label='Reaction Time (ms)', alpha=0.8)

ax.set_xlabel('Number of Strokes')
ax.set_ylabel('Values')
ax.set_title('Average Arousal (scaled) and Reaction Time by Stroke Count')
ax.set_xticks(x)
ax.set_xticklabels(stroke_grouped['CHINESE_STROKE'])
ax.legend()

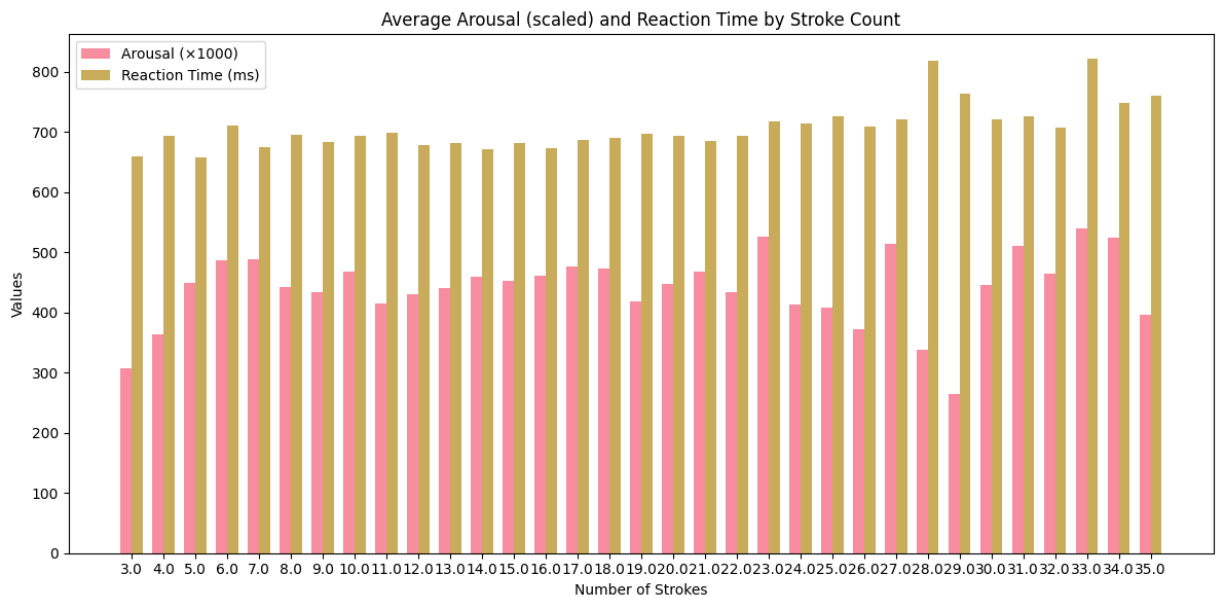
plt.tight_layout()
plt.show()
```

Task 3a

Average arousal and RT by stroke count:

	CHINESE_STROKE	ENGLISH_AROUSAL_MEAN	CHINESE_RT_MEAN
0	3.0	0.306500	659.623373
1	4.0	0.363444	692.974473
2	5.0	0.448556	657.017944
3	6.0	0.486000	711.511801
4	7.0	0.489280	674.809692
5	8.0	0.442310	695.775756
6	9.0	0.433063	682.889665
7	10.0	0.467480	692.990470
8	11.0	0.415250	698.790181
9	12.0	0.431053	677.933616
10	13.0	0.441075	682.287934
11	14.0	0.459811	670.896336
12	15.0	0.452443	682.323811
13	16.0	0.461571	672.410528
14	17.0	0.476438	687.281609
15	18.0	0.473397	689.994901
16	19.0	0.418784	696.905881
17	20.0	0.446883	693.399333
18	21.0	0.467172	685.501308
19	22.0	0.434643	693.527734
20	23.0	0.525778	717.135884
21	24.0	0.413333	714.783049
22	25.0	0.408000	726.890025
23	26.0	0.372250	709.292508
24	27.0	0.513500	720.417238
25	28.0	0.338000	818.280930
26	29.0	0.265333	763.931175
27	30.0	0.446333	720.898047
28	31.0	0.511000	726.330641
29	32.0	0.464000	708.179750
30	33.0	0.539000	821.360714
31	34.0	0.524500	747.662750
32	35.0	0.396000	760.532105

Task 3b



```
In [7]: # Task 4a: Import frequency data and merge
freq_data = pd.read_csv('cmn-opensubtitles-freq.tsv', sep='\t')
print(f"Frequency data loaded: {freq_data.shape}")

# Transform to log frequencies (assuming second column contains raw frequency)
freq_data['LOG_FREQ'] = np.log(freq_data.iloc[:, 1] + 1)

# First, identify the Chinese lemma column in your merged data
print("Available columns in merged data:")
print(list(merged_data.columns))
print("\nColumns that might contain Chinese lemmas:")
chinese_cols = [col for col in merged_data.columns if 'CHINESE' in col.upper]
print(chinese_cols)
chinese_lemma_col = 'CHINESE'
# Merge with existing data
merged_data = pd.merge(merged_data, freq_data, left_on=chinese_lemma_col,
                       right_on=freq_data.columns[0], how='left')

print(f"Data with frequencies: {merged_data.shape}")
```

Frequency data loaded: (184718, 2)

Available columns in merged data:

```
['CONCEPTICON_ID', 'CONCEPTICON_GLOSS_x', 'CHINESE', 'CHINESE_RT_MEAN', 'CHINESE_RT_ZSCORE', 'CHINESE_STROKE', 'LINE_IN_SOURCE_x', 'CONCEPTICON_GLOSS_y', 'ENGLISH', 'ENGLISH_VALENCE_MEAN', 'ENGLISH_AROUSAL_MEAN', 'ENGLISH_DOMINANCE_MEAN', 'LINE_IN_SOURCE_y', 'stroke_quintile', 'arousal_quintile', 'rt_quintile']
```

Columns that might contain Chinese lemmas:

```
['CHINESE', 'CHINESE_RT_MEAN', 'CHINESE_RT_ZSCORE', 'CHINESE_STROKE']
```

Data with frequencies: (957, 19)

```
In [8]: # Task 4b: Explore frequency vs reaction time
print("\nTask 4b: Exploring frequency vs reaction time relationship")

# Check for missing values and create clean data for analysis
print(f"Missing values - LOG_FREQ: {merged_data['LOG_FREQ'].isna().sum()}")
print(f"Missing values - CHINESE_RT_MEAN: {merged_data['CHINESE_RT_MEAN'].isna().sum()}")

# Create a clean dataset with no missing values for the plot
plot_data = merged_data[['LOG_FREQ', 'CHINESE_RT_MEAN']].dropna()
print(f"Valid data points for plotting: {len(plot_data)}")

if len(plot_data) < 2:
    print("Error: Not enough valid data points for analysis")
else:
    # Create scatter plot with marginals
    fig, axes = plt.subplots(2, 2, figsize=(10, 8),
                             gridspec_kw={'height_ratios': [1, 3], 'width_ratios': [1, 3]})

    # Main scatter plot
    axes[1,0].scatter(plot_data['LOG_FREQ'], plot_data['CHINESE_RT_MEAN'],
                      alpha=0.6, s=30)
    axes[1,0].set_xlabel('Log Frequency')
    axes[1,0].set_ylabel('Reaction Time (ms)')
    axes[1,0].set_title('Reaction Time vs Log Frequency')
```

```
# Add trend line only if we have enough data points
if len(plot_data) >= 2:
    try:
        z = np.polyfit(plot_data['LOG_FREQ'], plot_data['CHINESE_RT_MEAN'], 1)
        p = np.poly1d(z)
        axes[1,0].plot(plot_data['LOG_FREQ'], p(plot_data['LOG_FREQ']),
                        'r--')
    except np.linalg.LinAlgError:
        print("Warning: Could not fit trend line due to numerical issues")

# Marginal histograms
axes[0,0].hist(plot_data['LOG_FREQ'], bins=30, alpha=0.7)
axes[0,0].set_title('Log Frequency Distribution')
axes[0,0].set_xlim(axes[1,0].get_xlim())

axes[1,1].hist(plot_data['CHINESE_RT_MEAN'], bins=30, alpha=0.7, orientation='vertical')
axes[1,1].set_title('RT Distribution')
axes[1,1].set_ylim(axes[1,0].get_ylim())

# Remove unused subplot
axes[0,1].remove()

plt.tight_layout()
plt.show()

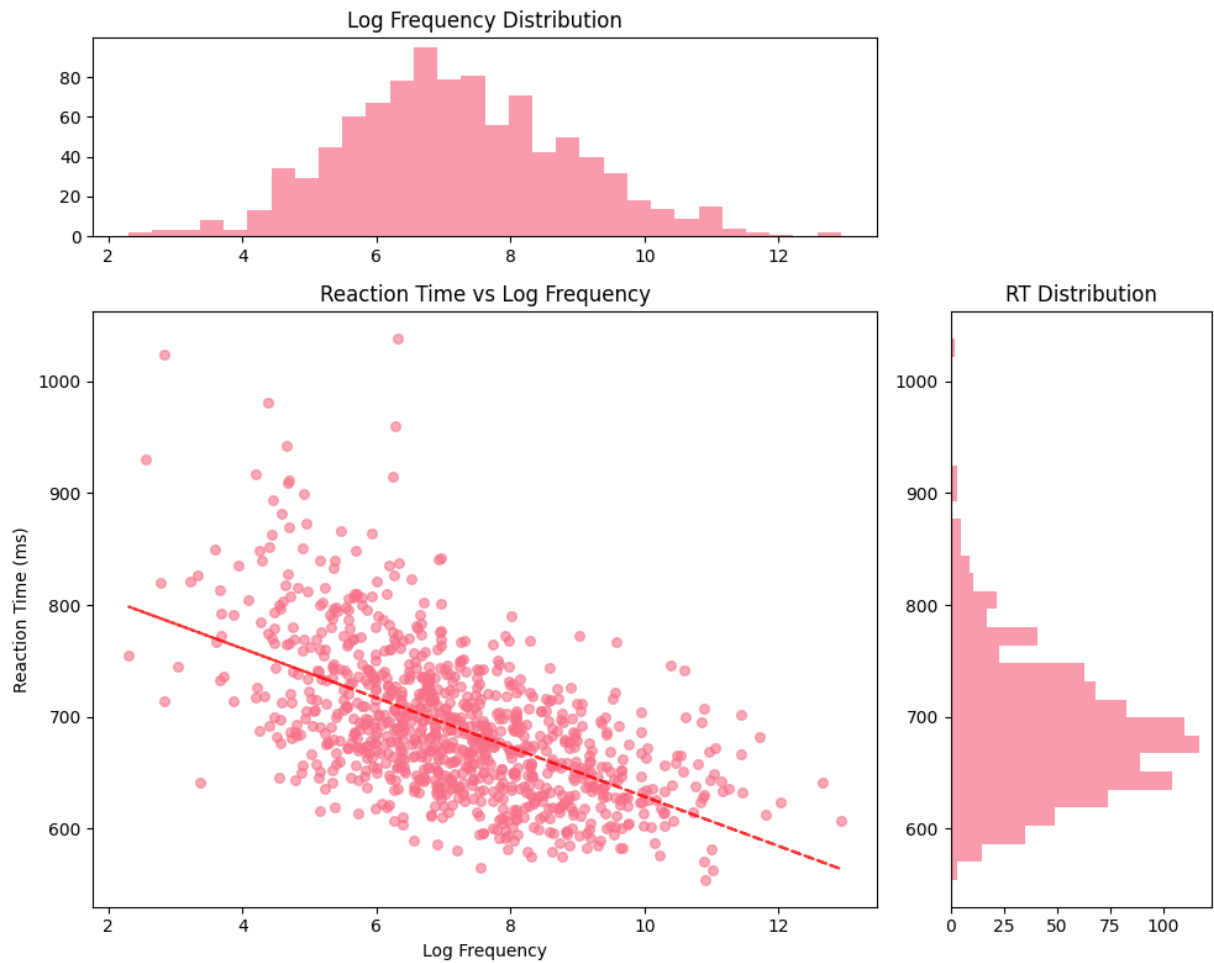
# Calculate correlation
freq_rt_corr = plot_data['LOG_FREQ'].corr(plot_data['CHINESE_RT_MEAN'])
print(f"Correlation between log frequency and reaction time: {freq_rt_corr}")
```

Task 4b: Exploring frequency vs reaction time relationship

Missing values - LOG_FREQ: 1

Missing values - CHINESE_RT_MEAN: 0

Valid data points for plotting: 956



Correlation between log frequency and reaction time: -0.564

```
In [9]: # Task 5a: Createing the frequency buckets

merged_data['freq_bucket'] = pd.qcut(merged_data['LOG_FREQ'], 5,
                                     labels=['Very Low', 'Low', 'Medium', 'Hig

print("Frequency bucket distribution:")
print(merged_data['freq_bucket'].value_counts().sort_index())
```

Frequency bucket distribution:

freq_bucket

Very Low 192

Low 191

Medium 191

High 191

Very High 191

Name: count, dtype: int64

```
In [10]: # Task 5b: Create correlation function
def compute_correlations(group):
    """Compute correlations between predictor variables and reaction times"""
    correlations = {
        'stroke_rt_corr': group['CHINESE_STROKE'].corr(group['CHINESE_RT_MEAN']),
        'arousal_rt_corr': group['ENGLISH_AROUSAL_MEAN'].corr(group['CHINESE_RT_MEAN']),
        'n_observations': len(group)
    }
    return pd.Series(correlations)
```

```
In [11]: # Task 5c: Applying the function across frequency bands
freq_band_correlations = merged_data.groupby('freq_bucket').apply(compute_correlations)
print("Correlations by frequency band:")
print(freq_band_correlations)

# Visualize the correlations by frequency band
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# Stroke-RT correlations
ax1.bar(range(len(freq_band_correlations)), freq_band_correlations['stroke_rt_corr'])
ax1.set_xlabel('Frequency Band')
ax1.set_ylabel('Correlation with RT')
ax1.set_title('Stroke Count - RT Correlation by Frequency Band')
ax1.set_xticks(range(len(freq_band_correlations)))
ax1.set_xticklabels(freq_band_correlations.index, rotation=45)
ax1.axhline(y=0, color='black', linestyle='--', alpha=0.3)

# Arousal-RT correlations
ax2.bar(range(len(freq_band_correlations)), freq_band_correlations['arousal_rt_corr'])
ax2.set_xlabel('Frequency Band')
ax2.set_ylabel('Correlation with RT')
ax2.set_title('Arousal - RT Correlation by Frequency Band')
ax2.set_xticks(range(len(freq_band_correlations)))
ax2.set_xticklabels(freq_band_correlations.index, rotation=45)
ax2.axhline(y=0, color='black', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
```

Correlations by frequency band:

freq_bucket	stroke_rt_corr	arousal_rt_corr	n_observations
Very Low	0.143329	0.067861	192.0
Low	0.121698	0.073189	191.0
Medium	0.065003	0.039538	191.0
High	0.021093	-0.003912	191.0
Very High	0.010031	0.054907	191.0

```
/tmp/ipykernel_8041/4243378226.py:2: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.
```

```
freq_band_correlations = merged_data.groupby('freq_bucket').apply(compute_correlations)
```

```
/tmp/ipykernel_8041/4243378226.py:2: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
```

```
freq_band_correlations = merged_data.groupby('freq_bucket').apply(compute_correlations)
```

